

Introduction to the Rust Programming Language

2026-10-01 — A Purdue Hackers workshop

Arhan Chaudhary

<https://github.com/ArhanChaudhary>

arhan.ch@gmail.com



Ferris, the Rust mascot

Outline

- 5min introduction
- 5min rust set-up
- 15min lecturing about basic Rust concepts
- 5min challenge problem
- 20min lecturing
- 5min challenge problem
- 10min lecturing
- 25min live coding
- Rest of the time: Rust exercises

What is Rust?

What is Rust?

Rust is a general-purpose programming language that empowers everyone to build reliable and efficient software.

— <https://rust-lang.org/>

What is Rust?

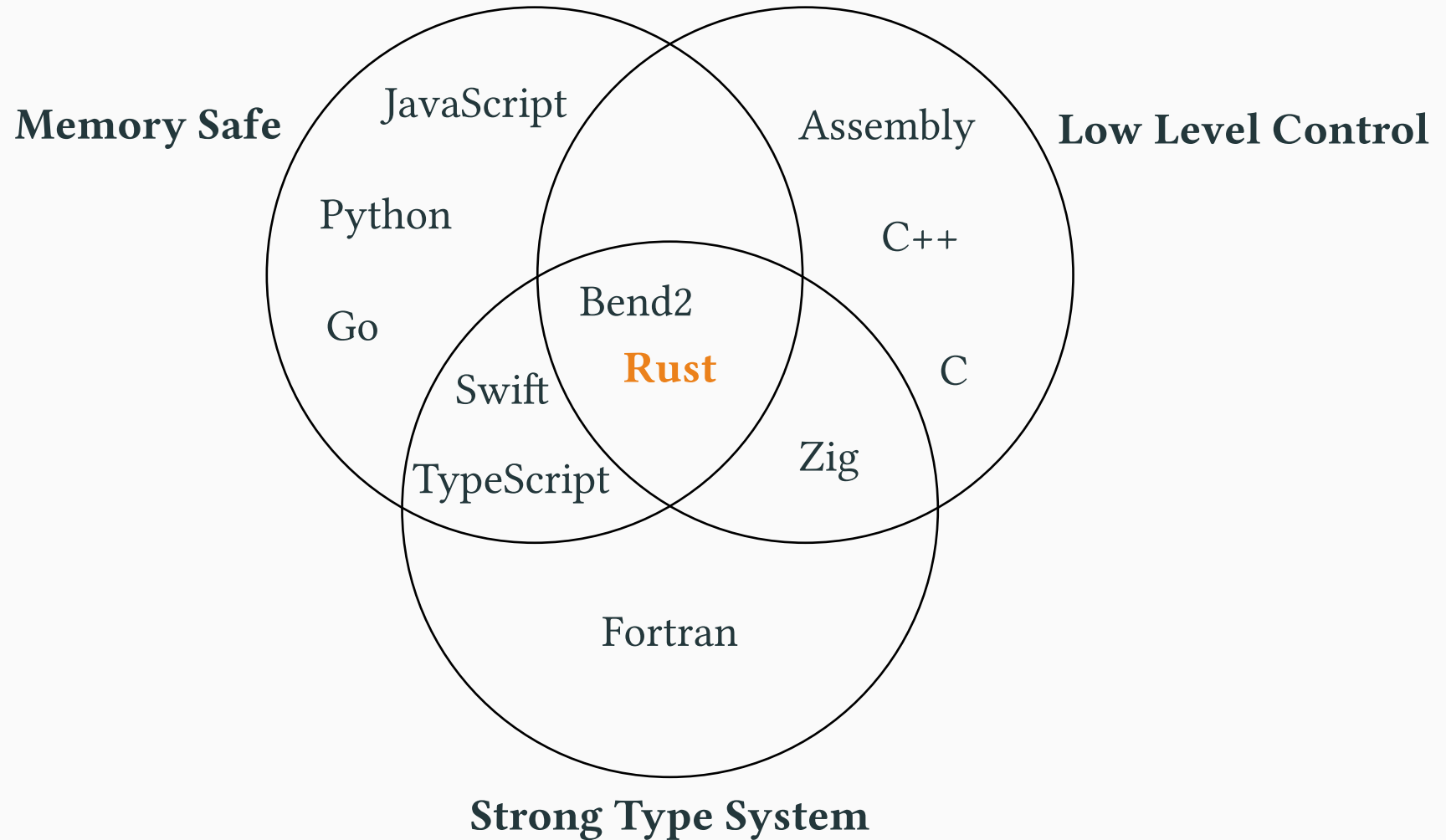
Rust is a general-purpose programming language that empowers everyone to build reliable and efficient software.

— <https://rust-lang.org/>

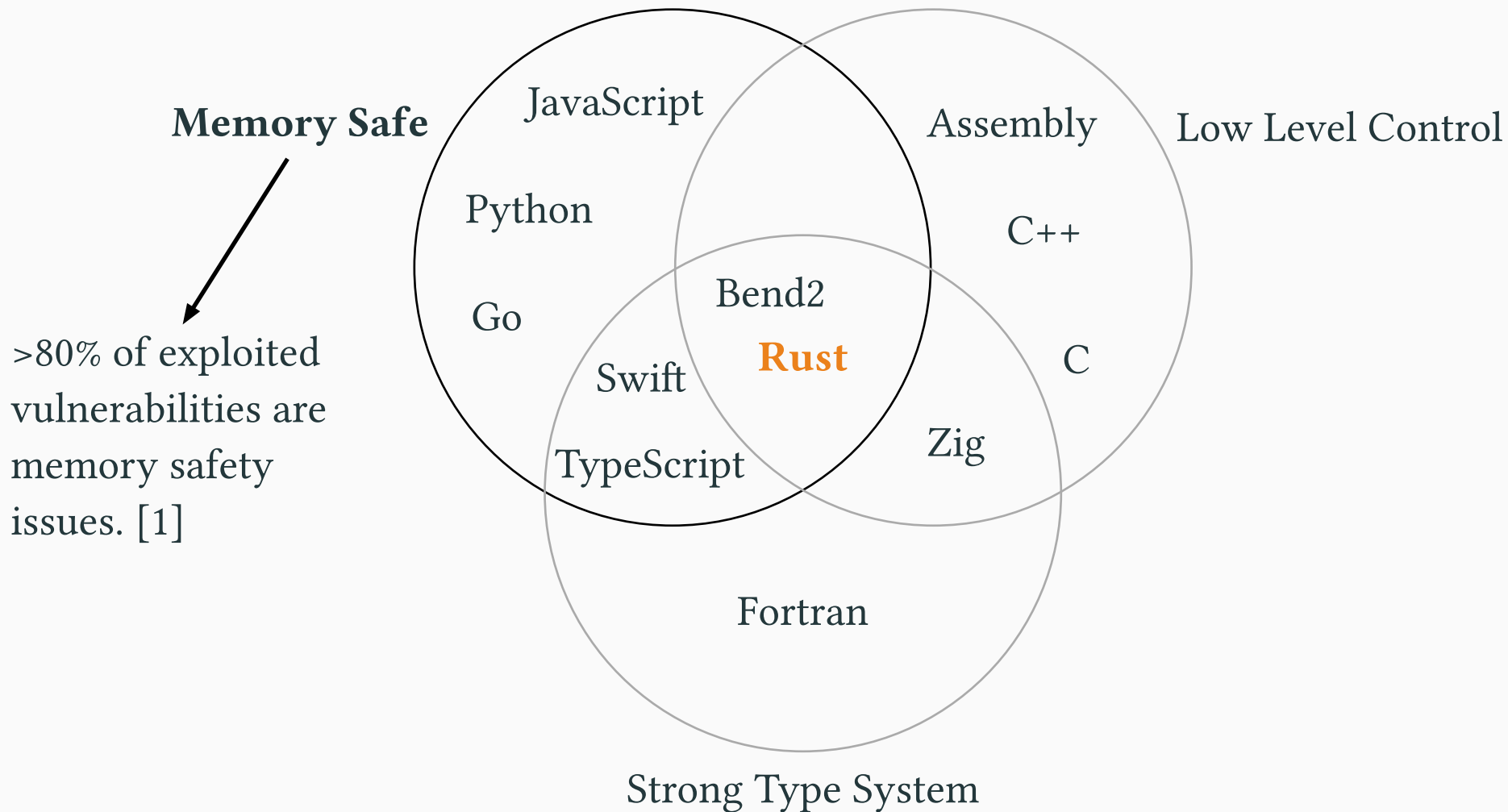
How?



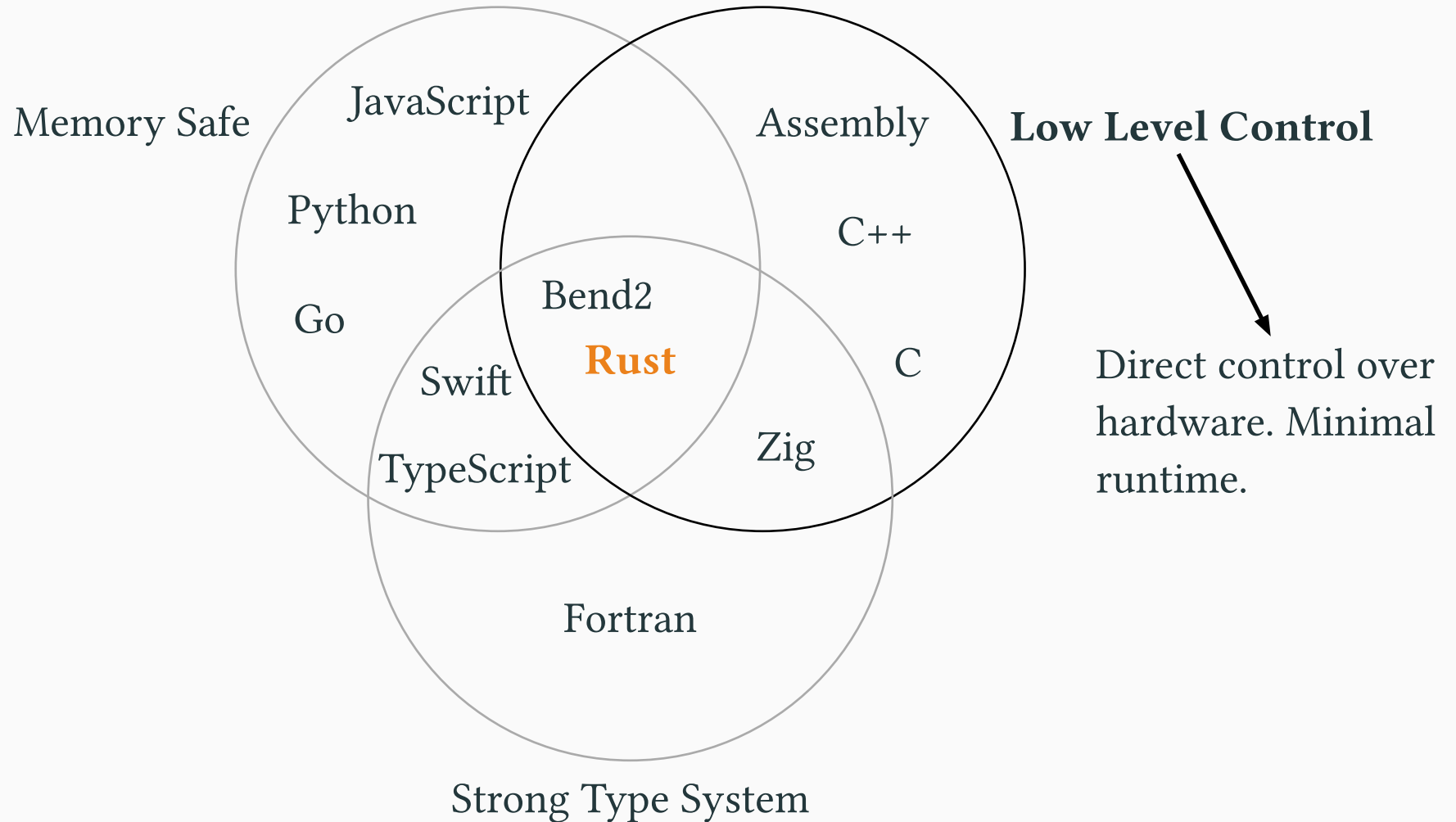
How is Rust empowering?



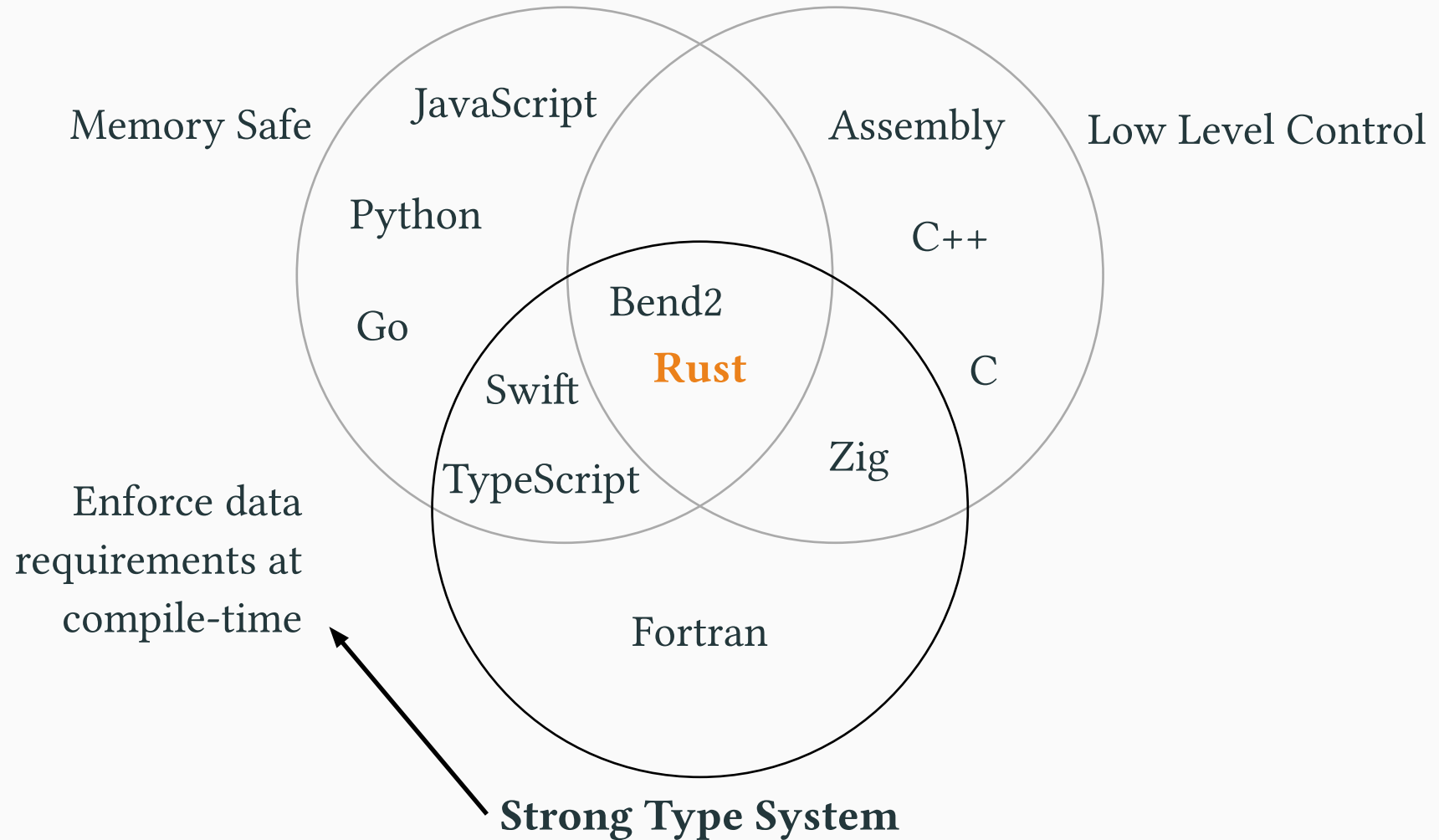
How is Rust empowering?



How is Rust empowering?

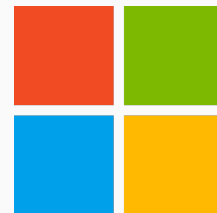
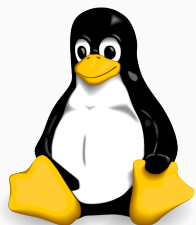


How is Rust empowering?



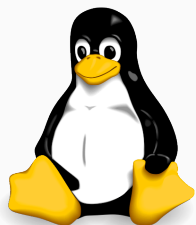
Who uses Rust?

You probably run Rust code every day



Who uses Rust?

You probably run Rust code every day



As do AI agents

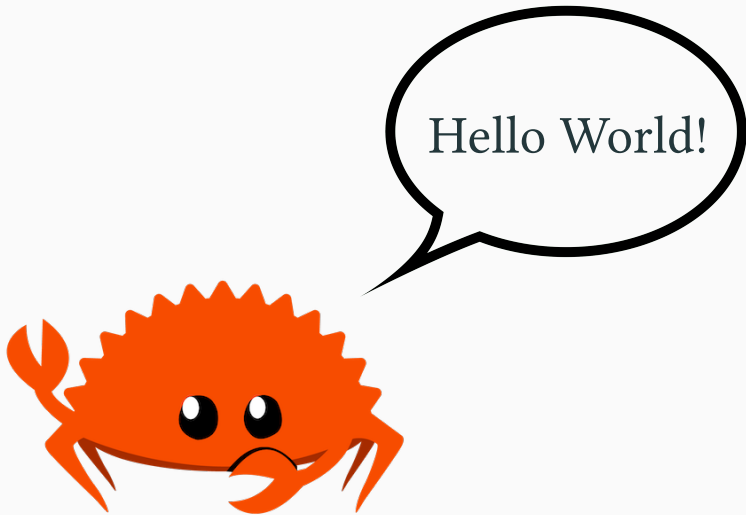


Let's learn some Rust!

<https://phack.rs/rs-2026>

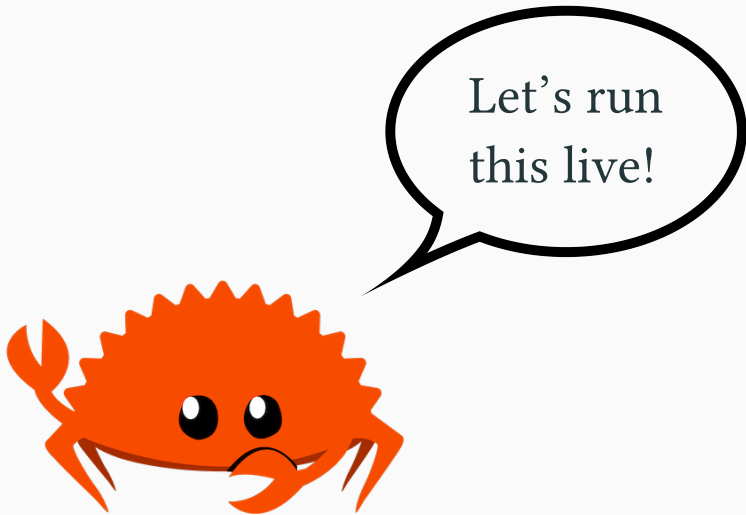
Hello World

```
fn main() {  
    println!("Hello World!");  
}
```



Hello World

```
fn main() {  
    println!("Hello World!");  
}
```



Variables

```
fn main() {  
    let power = 1000;  
    println!("The power level is: {power}");  
  
}
```

Variables

```
fn main() { // this creates a scope
    let power = 1000;
    println!("The power level is: {power}");
}
```

Variables are only valid inside scopes (curly braces)

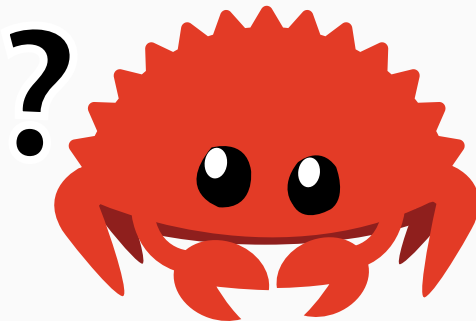
Variables

```
fn main() {  
    let power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```

Variables

```
fn main() {  
    let power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```

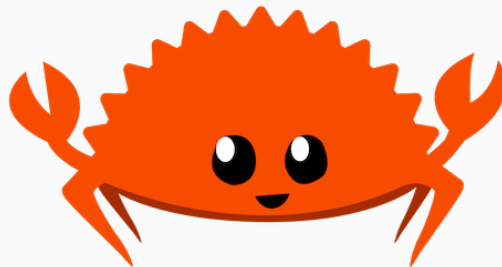
error[E0384]: cannot assign twice to immutable variable `power`



Variables

```
fn main() {  
    let mut power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```

- Variables are *immutable* (cannot change) by default
- *mutability* (can change) is opt-in



Variables

```
fn data_types() {  
    // Every variable has a type  
    // `integer` has the integer type `i32`  
    let integer = 5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a type  
    // `integer` has the integer type `i32`  
    let integer = 5;  
    // `integer2` is optionally annotated explicitly  
    let integer2: i32 = 5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a type  
    // `integer` has the integer type `i32`  
    let integer = 5;  
    // `integer2` is optionally annotated explicitly  
    let integer2: i32 = 5;  
    let true_or_false: bool = true;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a type  
    // `integer` has the integer type `i32`  
    let integer = 5;  
    // `integer2` is optionally annotated explicitly  
    let integer2: i32 = 5;  
    let true_or_false: bool = true;  
    let decimal: f64 = 4.5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a type  
    // `integer` has the integer type `i32`  
    let mut integer = 5;  
    // `integer2` is optionally annotated explicitly  
    let integer2: i32 = 5;  
    let true_or_false: bool = true;  
    let decimal: f64 = 4.5;  
  
    integer = decimal; // different types  
}
```

error[E0308]: mismatched types

```
fn main() {  
    // Reads a user input number from the terminal  
    let integer: i32 = utils::read_number();  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let integer: i32 = utils::read_number();  
    if integer == 42 {  
        println!("42 is the meaning of life");  
    }  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let integer: i32 = utils::read_number();  
    if integer == 42 {  
        println!("42 is the meaning of life");  
    }  
    let abs = if integer < 0 {  
        -integer  
    } else {  
        integer  
    };  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let mut integer: i32 = utils::read_number();  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let mut integer: i32 = utils::read_number();  
    // If `integer` is positive, let's  
    // subtract one until `integer` is zero.  
    while integer != 0 {  
        println!("Count down: {integer}");  
        integer -= 1;  
    }  
}
```


Ownership can be confusing!
Don't be afraid to ask questions!

How do programming languages manage their memory?

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!
 - The rules manage memory automatically **without overhead**

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!
 - The rules manage memory automatically **without overhead**
 - Memory unsafe $\Rightarrow$ the rules are violated $\Rightarrow$ your program won't compile

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

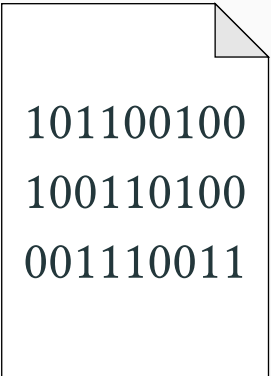
“Purdue Hackers” is hardcoded into the program executable

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable



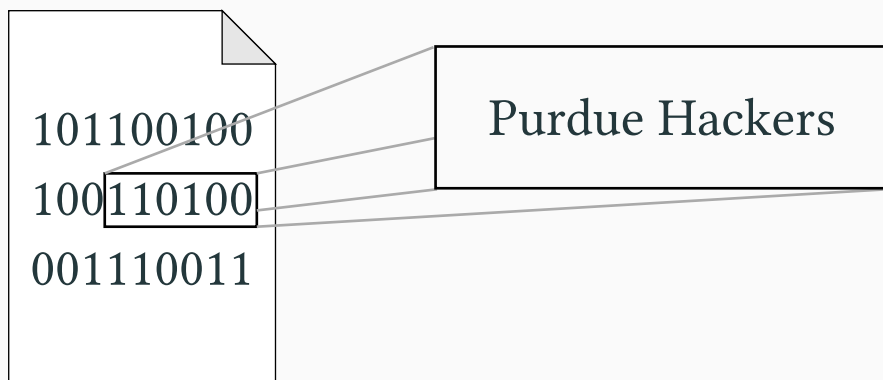
```
101100100  
100110100  
001110011
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable

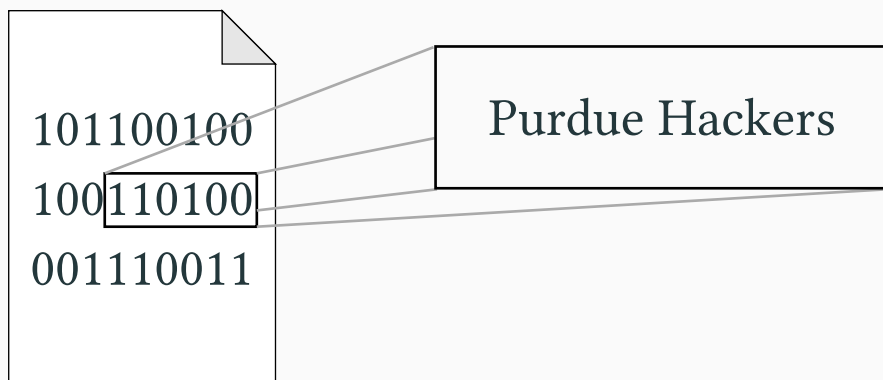


Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable



Let's prove it!

```
xxd ~/purdue-hackers | less
```

Ownership

Let's focus on a very common data structure: strings

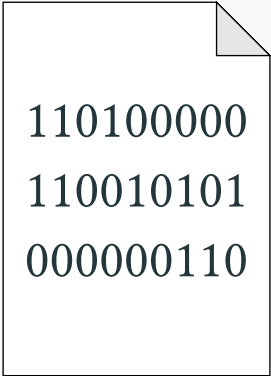
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```

Input

A box with a folded top-right corner containing three lines of binary code.

```
110100000  
110010101  
000000110
```

Ownership

Let's focus on a very common data structure: strings

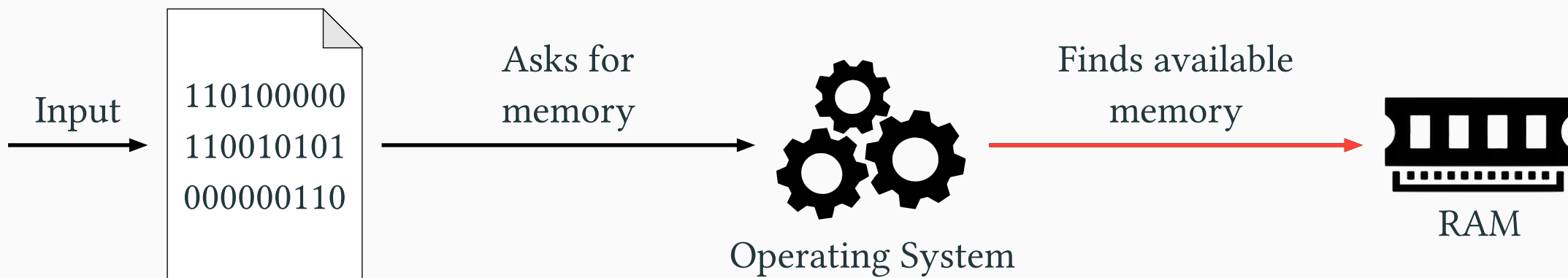
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

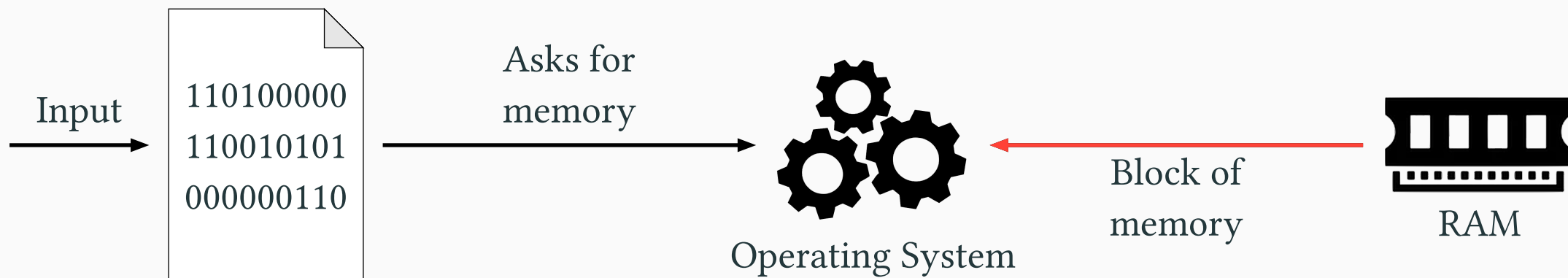
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a

Type	Value
address	0x557593ec3d60
length	5

Memory address	Value
0x557593ec3d60	H
0x557593ec3d61	e
0x557593ec3d62	l
0x557593ec3d63	l
0x557593ec3d64	o

Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a

Type	Value
address	0x557593ec3d60
length	5

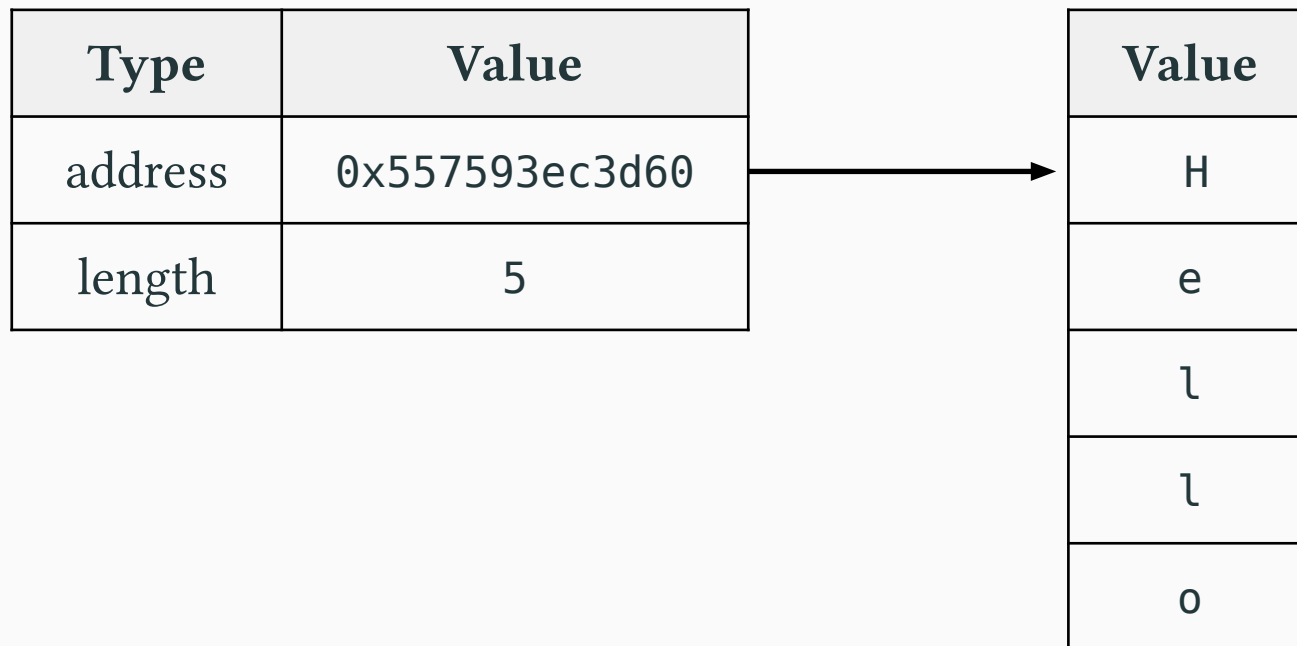


Memory address	Value
0x557593ec3d60	H
0x557593ec3d61	e
0x557593ec3d62	l
0x557593ec3d63	l
0x557593ec3d64	o

Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a



Ownership

How do Strings interact with scopes?

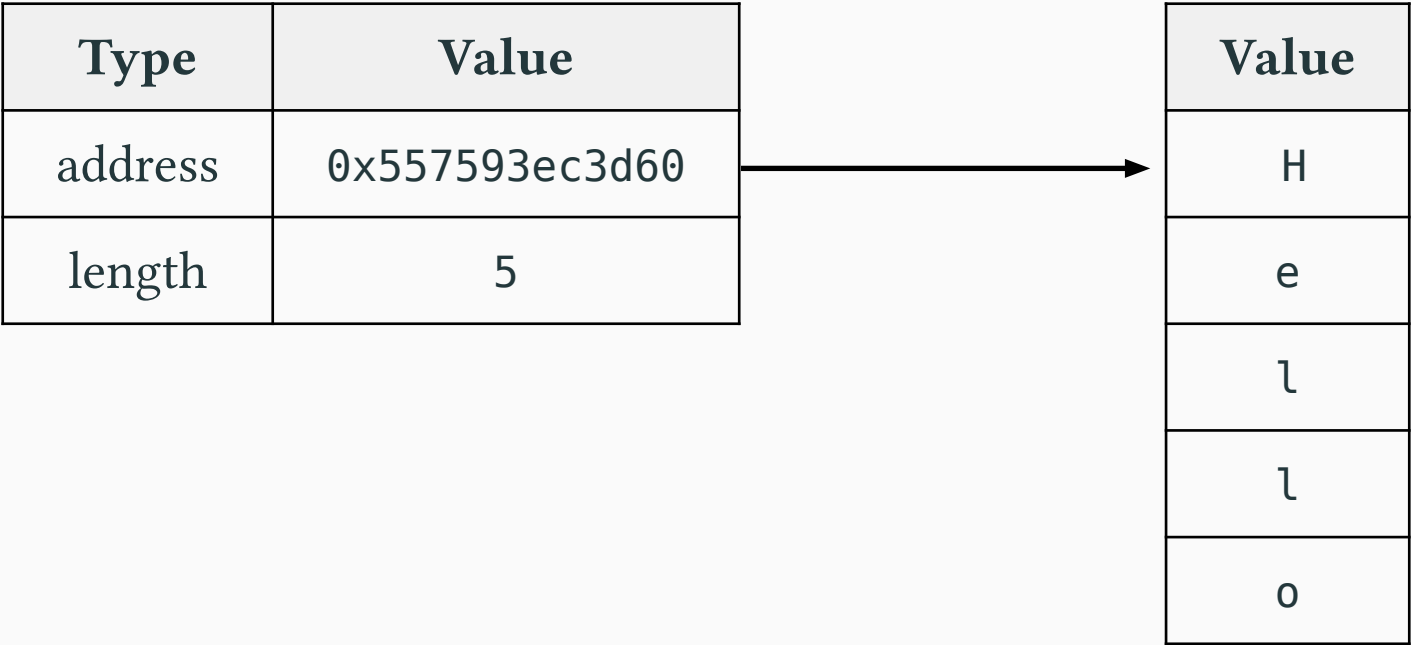
```
fn main() {  
    {  
        // `a` is created inside a scope  
        let a: String = utils::read_string();  
        // `a` is used inside a scope  
        println!("{a} is awesome!");  
    }  
    // `a` has left its scope  
}
```

Ownership

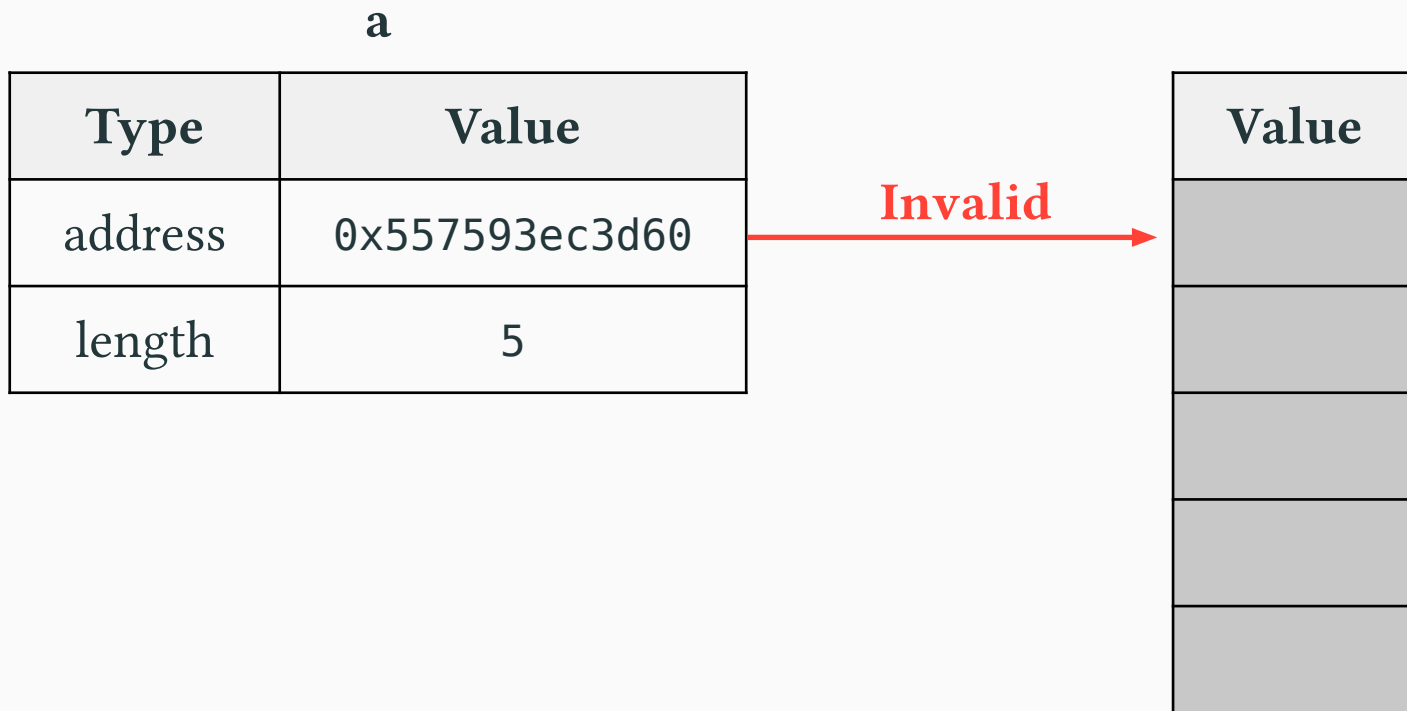
How do Strings interact with scopes?

```
fn main() {  
    {  
        // `a` is created inside a scope  
        let a: String = utils::read_string();  
        // `a` is used inside a scope  
        println!("{a} is awesome!");  
    }  
    // `a` has left its scope  
    // What happened to the memory?  
}
```

a



Value
H
e
l
l
o



Remember: Rust manages memory automatically

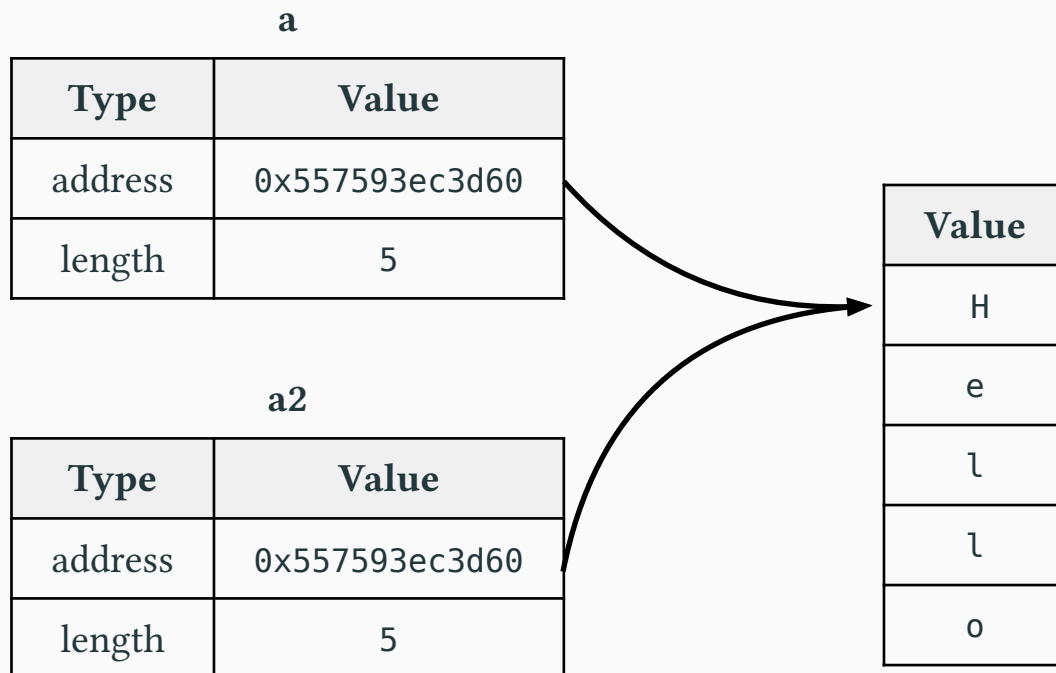
- Rust inserts code to free up memory back to the operating system

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

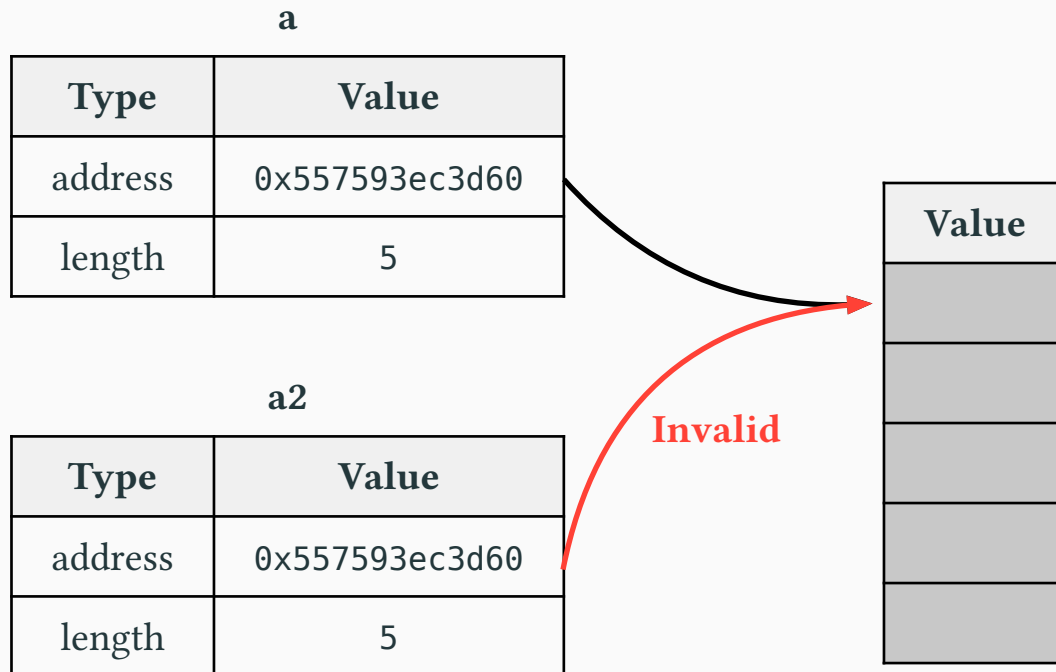
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



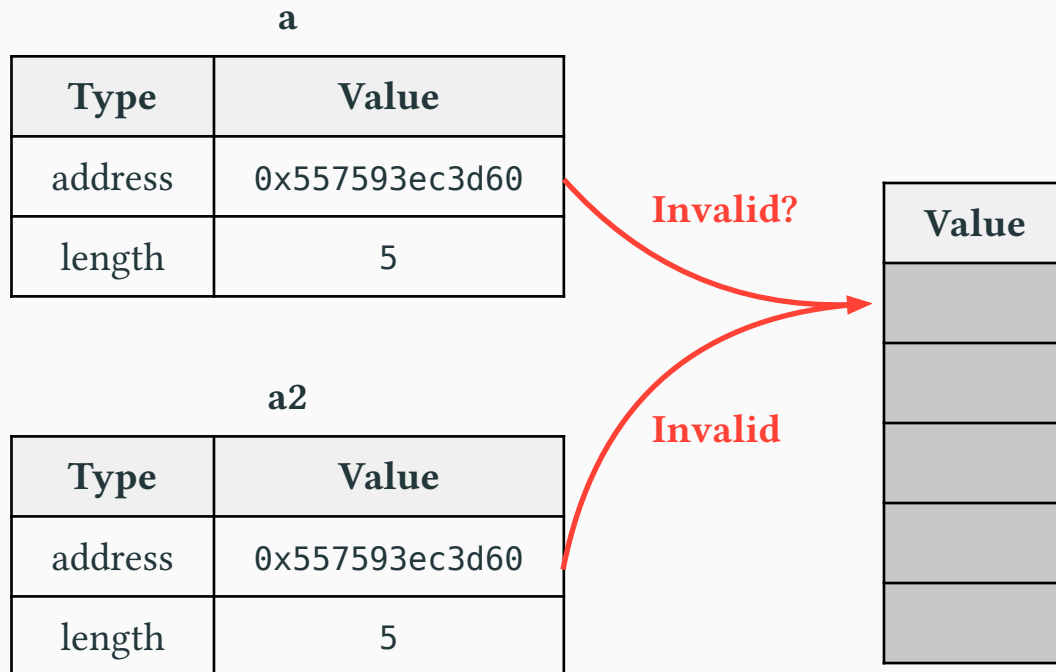
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



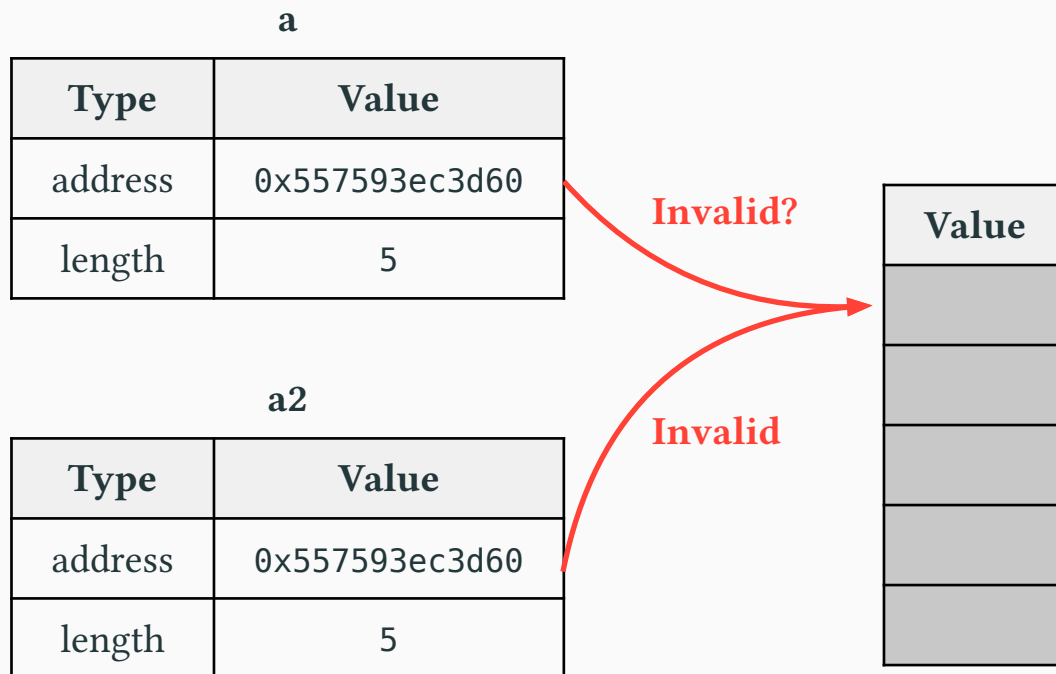
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

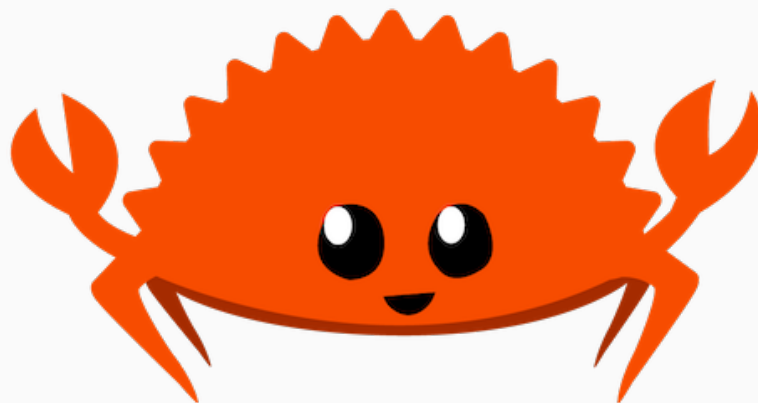


This is a memory safety bug!
This leads to undefined behavior!

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

error[E0382]: borrow of moved value: `a`



What are the rules of ownership?

What are the rules of ownership?

- 1) Each value in Rust has an owner

What are the rules of ownership?

- 1) Each value in Rust has an owner
- 2) There can only be one owner at a time

What are the rules of ownership?

- 1) Each value in Rust has an owner
- 2) There can only be one owner at a time
- 3) When the owner goes out of scope, the value will be cleaned up

Ownership

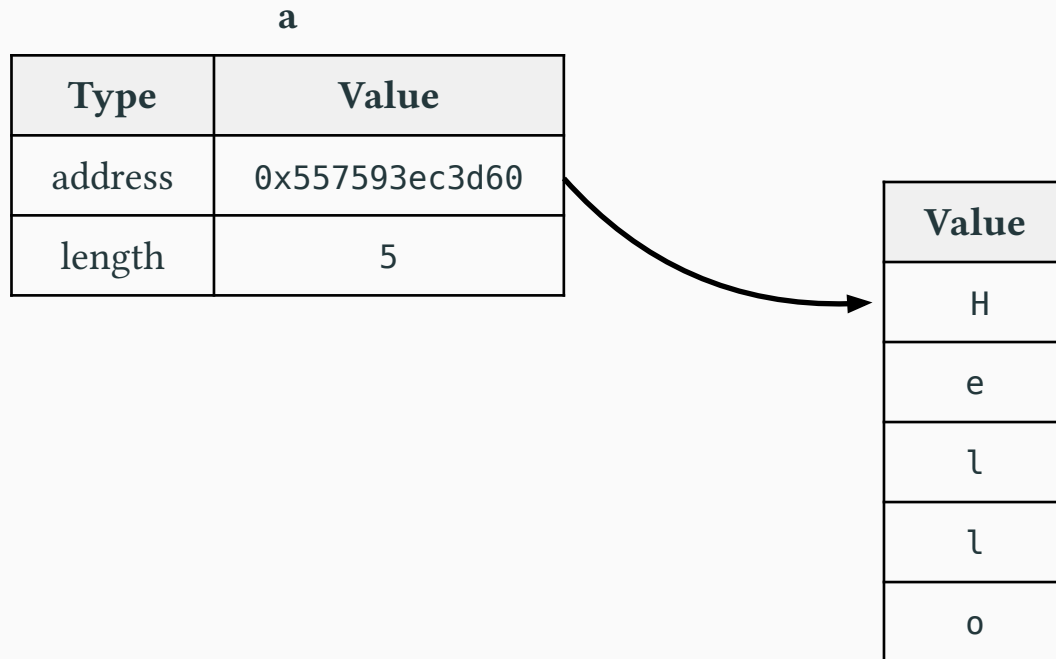
```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

How does this program violate the rules of ownership?

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

1) Each value in Rust has an owner



Ownership

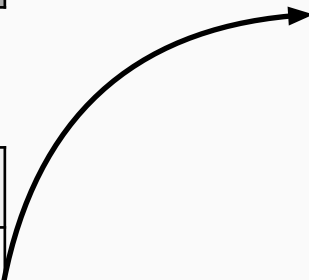
```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

2) There can only be one owner at a time

a	
Type	Value
address	0x557593ec3d60
length	5

a2	
Type	Value
address	0x557593ec3d60
length	5

Value
H
e
l
l
o



Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

3) When the owner goes out of scope, the value will be cleaned up

a	
Type	Value
address	0x557593ec3d60
length	5

a2	
Type	Value
address	0x557593ec3d60
length	5

Invalid

Value

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

Variable used after value was moved

a	
Type	Value
address	0x557593ec3d60
length	5

a2	
Type	Value
address	0x557593ec3d60
length	5

Invalid

Value

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    foo(a);  
    println!("{a}")  
}  
  
fn foo(s: String) {  
    println!("In function: {s}");  
}
```

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    foo(a);  
    println!("{a}")  
}  
  
fn foo(s: String) {  
    println!("In function: {s}");  
}
```

error[E0382]: borrow of moved value: `a`

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2 = foo(a);  
    println!("{a2}")  
}  
  
fn foo(s: String) -> String {  
    println!("In function: {s}");  
    // Return the variable `s`  
    s  
}
```

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2 = foo(a);  
    println!("{a2}")  
}  
  
fn foo(s: String) -> String {  
    println!("In function: {s}");  
    // Return the variable `s`  
    s  
}
```

This is too verbose!

Borrowing

```
fn main() {  
    let a: String = utils::read_string();  
    foo(&a);  
    println!("{a}")  
}  
  
fn foo(s: &String) {  
    println!("In function: {s}");  
}
```

- `&a` *borrow*s the value from the owner

Borrowing

```
fn main() {  
    let a: String = utils::read_string();  
    foo(&a);  
    println!("{a}")  
}  
  
fn foo(s: &String) {  
    println!("In function: {s}");  
}
```

- `&a` *borrow*s the value from the owner
 - Immutable borrow (**Read-only**)

Borrowing

```
fn main() {  
    let a: String = utils::read_string();  
    foo(&a);  
    println!("{a}")  
}  
  
fn foo(s: &String) {  
    println!("In function: {s}");  
}
```

- `&a` *borrow*s the value from the owner
 - Immutable borrow (**Read-only**)
 - What if we wanted to borrow, but write to the value?

Borrowing

```
fn main() {  
    let mut a: String = utils::read_string();  
    foo(&mut a);  
    println!("{a}")  
}  
  
fn foo(s: &mut String) {  
    println!("In function: {s}");  
  
}
```

Borrowing

```
fn main() {  
    let mut a: String = utils::read_string();  
    foo(&mut a);  
    println!("{a}")  
}  
  
fn foo(s: &mut String) {  
    println!("In function: {s}");  
    // Append an exclamation mark to the end  
    s.push('!');  
}
```

Borrowing

```
fn main() {  
    let mut a: String = utils::read_string();  
    foo(&mut a);  
    println!("{a}")  
}  
  
fn foo(s: &mut String) {  
    println!("In function: {s}");  
    // Append an exclamation mark to the end  
    s.push('!');  
}
```

- `&mut a` mutably borrows the value (**Read and write**)

Borrowing

```
fn main() {  
    let a: String = utils::read_string();  
  
    let b: &String = &a;  
    let c: &String = &a;  
    let d: &String = &a;  
  
    // ... a bunch of complicated code  
  
    println!("{b} {c} {d}");  
}
```

Borrowing

```
fn main() {  
    let mut a: String = utils::read_string();  
  
    let b: &String = &a;  
    let c: &String = &a;  
    let d: &mut String = &mut a;  
  
    // ... a bunch of complicated code  
  
    println!("{b} {c} {d}");  
}
```

Borrowing

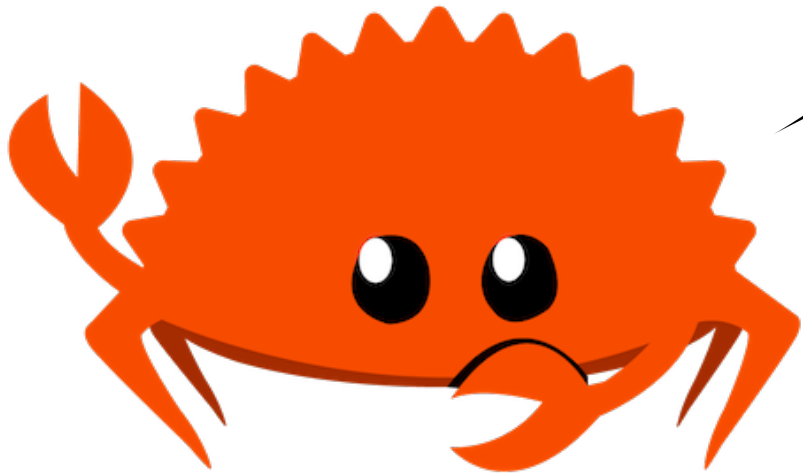
```
fn main() {  
    let mut a: String = utils::read_string();  
  
    let b: &String = &a;  
    let c: &String = &a;  
    let d: &mut String = &mut a;  
  
    // ... a bunch of complicated code  
  
    println!("{b} {c} {d}");  
}
```

error[E0502]: cannot borrow `a` as mutable because it is also borrowed as immutable

Let's do some live coding!

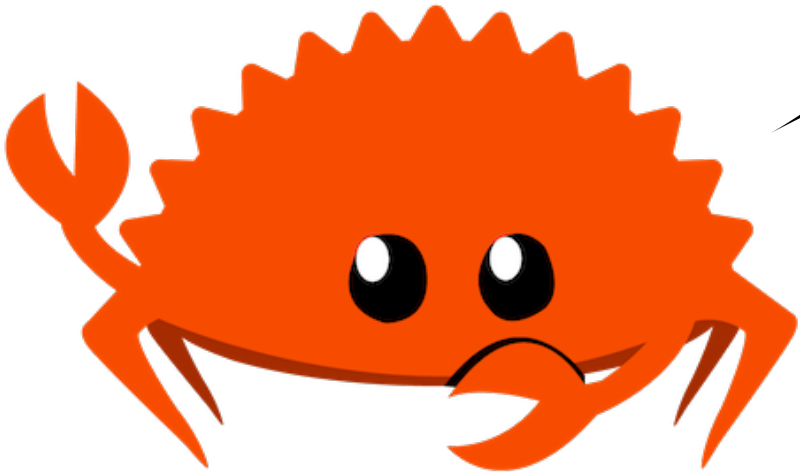
Further Learning

- *The Rust Programming Language* book (doc.rust-lang.org/stable/book)
 - How I learned Rust
 - We roughly covered the first 6 out of 21 chapters in this workshop
- Rustlings (rustlings.rust-lang.org)
 - Complementary learning exercises as you read the book



Thank you
for listening!

Let's do some
exercises!



Bibliography

- [1] “What is memory safety and why does it matter?,” *Internet Security Research Group*. [Online]. Available: <https://www.memorysafety.org/docs/memory-safety/>